

A Time Series Self-Supervised Contrastive Pretraining Method With Data Augmentation Using Discrepancy of Reconstruction Information Loss

Zhen Zhang[✉], Yongming Han[✉], *Member, IEEE*, Bo Ma[✉], and Zhiqiang Geng[✉]

Abstract—Self-supervised pretraining has shown considerable advantages in multiple time series tasks. A widely used class of methods is based on contrastive learning and a key problem is how to construct augmented samples. At present, explicit augmentation approaches are the mainstream for constructing positive pairs with similar features. However, explicitly augmented samples may deviate from the original data space and have aliasing features. Therefore, this article designs an end-to-end time series self-supervised contrastive pretraining network with implicit data augmentation using discrepancy of reconstruction information loss (CLR-DRL). Specifically, augmented positive pairs with discrepancies and inherent features of the original series can be generated automatically using different autoencoders (AE) due to different information loss in their hidden spaces. Combined with the joint constraint, inherent features can be extracted and the aliasing impact existing in explicit augmented approaches could be mitigated. Moreover, theoretical analysis is provided using gradient flow calculation, which can also be confirmed by experiment analysis. Experimentally, the CLR-DRL achieves state-of-the-art (SOTA) performance compared to current baselines on in- and cross-domain settings on nine real-world benchmarks for both time series anomaly detection (AD) and classification tasks. An actual pipeline transportation status monitoring case is provided to verify the effectiveness.

Index Terms—Autoencoder (AE), data augmentation, industrial process, self-supervised learning (SSL), time series.

I. INTRODUCTION

TIME series are widely existed in real-world scenarios, including human activity recognition (HAR) [1], industrial anomaly detection (AD) [2], and energy consumption prediction [3]. The key to successfully handling these tasks is to achieve effective modeling of time series and to identify inherent features implicit in data. In recent years, research on achieving high-precision deep learning-based time series

modeling using massive amounts of data with labels has shown a promising progress [4]. However, in a real-world industrial scenario, there are some problems using the above approach: 1) the data for a new industrial scenario is scarce and 2) labeling a large number of time series is difficult, and only a few samples in real-world scenarios have been labeled [5].

To solve the problem of scarcity of labeled samples, researchers introduced the self-supervised learning (SSL) [6], which extracts inherent representations in the data through training on an upstream task without labeling samples, and uses them for target tasks. In recent years, self-supervised pretraining of time series using the contrastive learning framework with data augmentation has gained widespread attention and achieved a promising progress. The basic principle is that the sample representation is obtained through the contrastive constraint after generating two different augmented samples. Chen et al. [7] proposed a simple contrastive learning framework for visual representation (SimCLR), for the first time showing that the combination of data augmentations played a key role in representations by contrastive learning. Subsequently, Ian Tang et al. [8] introduced the SimCLR into the time series field and Eldele et al. [9] innovated the framework through cross-view forecasting tasks. In addition, the researchers further extended the modeling process by fusing the frequency domain information. Zhang et al. [6] designed a time–frequency consistency (TF-C) module and data augmentation approaches for frequency domain data to extract the representation of time series by pulling closer the time- and frequency-domain representations of the hidden space.

Although the above methods have achieved good performance in extracting time series representations, the data augmentation approaches they used are all explicit approaches, such as random flipping, jittering, and scaling, which are also widely used in the image processing field. However, different from discrete image data, which tends to contain more explicit and complete information, continuous time series data tends to contain more complex information. For example, as shown in Fig. 1(a), a sample labeled cat can still be considered a cat after being flipped. But for the time series shown in Fig. 1(b), after random flipping, a rising sample becomes a falling sample, which usually indicates two completely opposite working conditions in an actual industrial process. Therefore, augmenting samples through explicit augmentation approaches may not be appropriate in time series tasks. Specifically, the possible

Received 14 August 2024; revised 21 November 2024; accepted 2 December 2024. Date of publication 5 March 2025; date of current version 20 March 2025. This work was supported by the National Natural Science Foundation of China under Grant 62373035 and Grant 62203038. The Associate Editor coordinating the review process was Dr. Ferdinanda Ponci. (*Corresponding authors: Yongming Han; Zhiqiang Geng.*)

Zhen Zhang, Yongming Han, and Zhiqiang Geng are with the College of Information Science and Technology, Beijing University of Chemical Technology, Beijing 100029, China, and also with the Engineering Research Center of Intelligent Process System Engineering (PSE), Ministry of Education of China, Beijing 100029, China (e-mail: zhang_zhen@buct.edu.cn; hanym@mail.buct.edu.cn; gengzhiqiang@mail.buct.edu.cn).

Bo Ma is with the College of Mechanical and Electrical Engineering, Beijing University of Chemical Technology, Beijing 100029, China (e-mail: mabo@mail.buct.edu.cn).

Digital Object Identifier 10.1109/TIM.2025.3548177

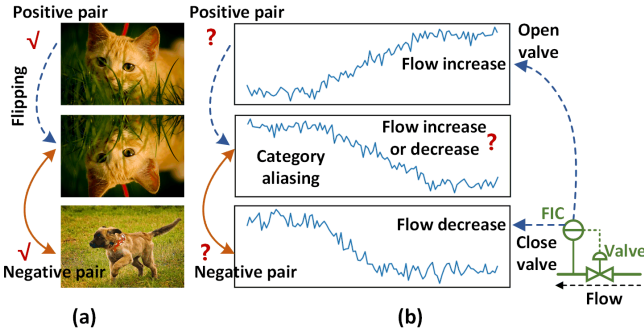


Fig. 1. Augmented samples by flipping operation of an (a) image and (b) industrial time series.

category aliasing of augmented samples has a large impact on industrial time-series classification and AD tasks.

In addition to the contrastive learning, the autoencoder (AE) is also an effective SSL method, which maps the times series to a hidden space to generate the effective representation. The training process of the AE is to minimize the error between the reconstructed and input time series. In this process, although there is information loss in the extracted representation, the reconstructed series can still be close to the original series [10]. Moreover, for different AEs, different reconstructed series can be obtained although the optimization objective is the same, which suggests that the representation obtained by the AE contains both the inherent feature and discrepant information. Therefore, it is natural to use the property of the AE that there is information loss in the reconstruction process to automatically generate augmentation samples required for contrastive learning.

Inspired by the above insights, this article incorporates the properties of the AE in the contrastive learning and proposes a time series self-supervised contrastive pretraining method with data augmentation using discrepancy of reconstruction information loss (CLR-DRL). Specifically, the CLR-DRL utilizes the discrepancies of the information loss in the hidden space of different AEs to obtain reconstructed series with discrepancies, and considers them as augmented samples needed for contrastive learning. Technically, the CLR-DRL does not use explicit augmentation technologies, but directly uses different AEs to obtain implicit augmented samples. Since both augmented samples come from the reconstruction process of the time series, the common features contained in the two augmented samples can be regarded as inherent representations. The CLR-DRL uses the contrastive loss to constrain the representations extracted from the two augmented samples to obtain the inherent representation of time series. Benefiting from the above design, the CLR-DRL can mitigate the categories aliasing problem caused by explicit augmentation approaches and obtain the inherent representation, making it achieving well performances in two target tasks with time series classification and AD.

Overall, our contributions can be summarized as follows.

- 1) Inspired by the discrepancies of the information loss in the reconstruction process of different AEs, a novel time series self-supervised contrastive pretraining method is proposed, which uses carefully designed AEs to generate

explicit augmented samples and extracts inherent representation.

- 2) Technically, the CLR-DRL is a simple but effective method, which generates samples and extracts inherent representations by a joint constraint covering the reconstructed and contrastive loss. Theoretically, the effectiveness of the CL-DRL is verified by gradient flow calculation of the joint constraint.
- 3) Experimentally, the CLR-DRL achieves state-of-the-art (SOTA) performance in two typical time series tasks with classification and AD, covering multiple domains of experimental settings. And extensive analysis experiments verify the proposed method.

Section II reviews the related work. Section III introduces the proposed CLR-DRL method. Section IV analyzes and discusses the performance of the CLR-DRL through extensive experiments. Section V verifies the application effect of the CLR-DRL using an actual industrial case. Section VI draws the conclusion of this article.

II. RELATED WORKS

A. Self-Supervised Contrastive Pretraining for Time Series

Currently, self-supervised pretraining has achieved great success in the fields of computer vision [11] and natural language processing [12]. Moreover, researchers have begun to study self-supervised pretraining methods for time series. Contrastive learning-based methods have made promising progress, which realizes time series representation by classifying positive and negative samples, and one of the keys is how to construct high-quality positive and negative pairs [6].

Franceschi et al. [13] considered subseries close or not close to the reference subseries as positive and negative samples, and used triplet loss for training to extract the representation. Yue et al. [14] called the above idea subseries consistency, and believed that it would cause the augmented samples to be susceptible to the “level shifts,” which could result in false positive pairs. To solve this problem, they proposed the time series to vector method (TS2Vec), which utilized context consistency to construct positive samples, arguing that only subseries with identical timestamps can be considered as positive pairs.

Different from the above method of using subseries to achieve negative sampling, inspired by the SimCLR [7], Ian Tang et al. [8] regarded the samples augmented by different explicit techniques in the entire series as positive samples, and other time series in a small batch as negative samples. They designed eight explicit time series augmented approaches, including random noise addition, scaling, and inverting. Recently, under the framework of the SimCLR, many methods with good performance have been proposed. Eldele et al. [9] designed a temporal and contextual contrastive module to implement time series modeling (TS-TCC) through cross-view prediction tasks, and achieved well performance in the time series classification and cross-domain transfer tasks. Furthermore, combining the frequency domain information of the time series, Yang and Hong [15] proposed the bilinear temporal-spectral fusion method (BTSF), which generated

instance-level positive samples in the time domain by random dropout and selected any one other series as a negative sample. And when modeling, the time- and frequency-domain features of the series were extracted through two encoders, and fused through a carefully designed fusion algorithm. Similarly, Woo et al. [16] also performed data augmentation in the time domain using explicit approaches, and then modeled the series by contrastive learning of disentangled seasonal-trend representations in time- and frequency-domain [contrastive learning of disentangled seasonal-trend representations (CoST)]. However, although the BTSF and the CoST incorporate frequency domain information in the modeling process, their data augmentation technologies are still oriented to time domain representation.

In order to directly utilize frequency domain information, Zhang et al. [6] proposed the TF-C method, which directly implemented data augmentation on the frequency domain, and designed three types of frequency domain augmentation techniques by artificially adding frequency domain perturbations. And the TF-C considered that the hidden representations in the time and frequency domain are close, so as to achieve a well representation. However, for the above contrastive learning methods, both time- and frequency-domain augmentations are all implemented by explicit approaches, making the augmented samples potentially category aliasing, which will lead to the representation that is, difficult to characterize the inherent features of the series.

B. Data Augmentation for Time Series

In current self-supervised contrastive pretraining methods, explicit augmentation methods are usually used to obtain the required augmented samples. For continuous time series data, explicit augmentation methods usually include jittering, scaling, inverting, reversing, shuffling, scrambling, and warping [17]. These methods are often randomly applied in time series. In addition to explicit methods, some works used neural network methods to implicitly implement data augmentation [18]. Shi et al. [19] used a temporal generative adversarial network (GAN) with a gradient penalty to generate samples to solve the data imbalance problem. Li et al. [20] proposed a bidirectional variational AE equipped with diffusion, denoise, and disentanglement for time series augmentation.

Compared with explicit augmentation methods, the method using neural networks for data augmentation can obtain richer sample types [18], and the samples are more closely related to the original sample due to the presence of a training process. Meanwhile, compared to the GAN and the diffusion methods that are more focused on generating new samples [21], the reconstructed series obtained by the compression and reconstruction of the AE will be more closely related to the original time series.

Based on the above review, the CLR-DLR naturally utilizes AEs to generate augmented samples required for contrastive learning. Specifically, since the contrastive learning method requires pairs of augmented series, the CLR-DRL uses two different AEs to generate the required samples. Meanwhile, since there are discrepancies in the samples reconstructed by different AEs, the common feature contained in two aug-

mented samples can be regarded as the inherent feature of the original series, which can be well extracted and characterized under the constraints of the contrastive loss.

III. METHOD

A. Problem Formulation

Refer to the definition of transfer learning [22] to formulate the problem. Given a source-domain dataset $D^{\text{Source}} = \{x_i^{\text{Source}} | i = 1, 2, \dots, U\}$ and a target-domain dataset $D^{\text{Target}} = \{(x_i^{\text{Target}}, y_i) | i = 1, 2, \dots, V\}$, D^{Source} and D^{Target} consists of unlabeled and labeled time series, respectively. And for time series $x \in \mathbb{R}^{C \times T}$ in D^{Source} and D^{Target} , C and T refer to the dimension and length of the time series, respectively. The label y in D^{Target} is different according to target tasks. Specifically, for the two application scenarios of classification and AD in this article, y can be defined as $y \in \{0, 1, \dots, Y\}$, where Y represents the number of categories in the classification scenario, and Y is 1 in the AD scenario, in which $Y = 1$ is marked as an anomaly.

The goal of self-supervised pretraining is that fine-tuning parameters of the model $F_{\text{pre}}(\cdot, \Theta)$ on D^{Target} containing V samples to obtain the model $F_{\text{tune}}(\cdot, \Phi)$, where $F_{\text{pre}}(\cdot, \Theta)$ is the pretrained model obtained on D^{Source} containing U samples, and then obtaining the representation $z_i^{\text{Target}} = F_{\text{tune}}(x_i^{\text{Target}}, \Phi)$ used for the target task, where $V \ll U$. Among the parameters, Θ and Φ are the parameters obtained through pretraining and fine-tuning, respectively. And for the model $F(\cdot)$, the process of $\Theta \rightarrow \Phi$ is fine-tuning.

B. Overall Architecture

The overall architecture of the CLR-DRL is shown in Fig. 2. First, the CLR-DRL utilizes two different AEs to generate two augmented series of the input series, and both of which could contain the inherent feature of the input series while differing under the reconstructed constraint. Then, the CLR-DRL performs feature extraction on two augmented samples through a shared weight feature extraction module. A joint constraint covering reconstruction and contrastive loss is imposed on the augmented samples and affine results of the two features, which leads the sample augmentation process and the representation extraction process to promote each other and makes the extracted representation can reflect the inherent feature of the input series. Specifically, two AEs and the feature extraction module of the CLR-DRL are carefully designed and implemented based on a graph neural network (GNN), a fully connected neural network (FNN), and a three-layer convolutional residual network (ResNet), which are shown in Fig. 2(c)–(e), respectively. The above process is self-supervised contrastive pretraining of the CLR-DRL and is performed in the source domain as shown in Fig. 2(a). After that, the pretrained model will be fine-tuned with the target domain data to conduct target tasks. During the fine-tuning process as shown in Fig. 2(b), time series is input into the pretrained CLR-DRL to extract the representation, which is then fed into the target model to realize the target task.

C. Data Augmentation

Given a time series x , the contrastive learning method first generates two augmented samples, x_s and x_w , through the

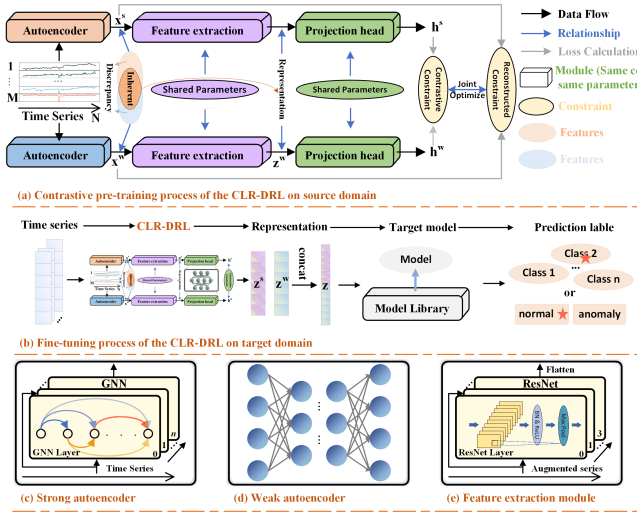


Fig. 2. Overall architecture of the CLR-DRL. (a) Contrastive pretraining process of the CLR-DRL on the source domain. (b) Fine-tuning process of the CLR-DRL on the target domain. (c) Strong AE. (d) Weak AE. (e) Feature extraction module.

augmentation approach. Then, in a small batch, contrastive learning attempts to maximize the similarity between x_s and x_w from the same sample, while minimizing the similarity between them and other samples. Therefore, two suitable augmentation samples play an important role in the performance of contrastive learning.

The properties of the AE in terms of the information loss and forcing the reconstructed series to tend toward the input series provide ideas for data augmentation. Therefore, two different AEs are used to obtain the augmented samples x_s and x_w in the CLR-DRL. Specifically, the input time series are reconstructed by a strong and a weak AE with different architectures to generate strong and weak augmented samples. In particular, a simple FNN is used as the weak AE, while a GNN is utilized to carefully design a strong AE that is, more suitable for temporal information. Following the convention in contrastive learning, the “strong” and “weak” are only used to distinguish between two enhancement approaches and do not indicate the reconstruction performance of the AE itself.

1) *Weak AE*: The weak AE is a simple three-layer FNN as shown in Fig. 2(d) which contains an input layer, an output layer, and a hidden layer. Given a time series $x \in \mathbb{R}^{C \times T}$, it is first mapped to a space with D_w dimension ($D_w \ll T$) and then reconstructed to the original space. In the hidden layer, it is transformed nonlinearly using a linear rectification function $\text{ReLU}(\cdot)$. So, the weak sample x_w can be calculated by

$$x_w = \text{ReLU}(x \cdot W_{w1}) \cdot W_{w2} \quad (1)$$

where $W_{w1} \in \mathbb{R}^{T \times D_w}$ and $W_{w2} \in \mathbb{R}^{D_w \times T}$ are the connection weight matrices between the input layer and the hidden layer, and the hidden layer and the output layer, respectively.

2) *Strong AE*: The strong AE is implemented by a unidirectional GNN as shown in Fig. 2(c). Referring to the forward propagation approach of time series information, a forward-connected graph structure is constructed. Specifically, the graph structure matrix $A \in \mathbb{R}^{T \times T}$ is fixed as an upper triangular matrix to constrain the direction of information

transmission so that it can only be propagated forward. Each element of A can be calculated by

$$A_{i,j} = \mathbf{1}_{i \leq j} \quad (2)$$

where $\mathbf{1}_{i \leq j}$ is an indicator function that equals to 1 when $i \leq j$, and otherwise 0. Then, for each layer of the strong AE, the temporal feature is modeled through the message-passing mechanism of the GNN. And each layer is connected by the residual connection. For the nodes of each layer, the calculation process is as follows:

$$H_i^n = \text{ReLU} \left(W_1^n \cdot H_i^{n-1} + W_2^n \cdot \sum_{j \in N(i)} A_{j,i} \cdot H_j^{n-1} \right) + H_i^{n-1} \quad (3)$$

where W_1 , W_2 , and H_i^n denote the weight matrices and the output of the node in the GNN layer, and $j \in N(i)$ denotes the neighbor nodes of the node i . Specifically, $H^0 \in \mathbb{R}^{C \times T}$ is the input time series x and $H_i^0 \in \mathbb{R}^{C \times 1}$ is the vector of the node i . The dimension of the weight matrices W_1^0 and W_2^0 is $\mathbb{R}^{D_s \times C}$, so the first layer will project the series to the space $\mathbb{R}^{D_s \times T}$ and the reminding layers encode in this space. Therefore, when $n > 1$, W_1^n and W_2^n belong to $\mathbb{R}^{D_s \times D_s}$ and H^n belongs to $\mathbb{R}^{D_s \times T}$.

After n -layer stacking, the strong sample x_s can be obtained by an affine transformation of the encoded output

$$x_s = W_{s1} \cdot H^n \quad (4)$$

where $W_{s1} \in \mathbb{R}^{C \times D_s}$ denotes the parameter matrix of the strong AE.

The above process of augmenting samples can be formulated by

$$x_s = \text{AE}_s(x, \theta_{A_s}), \quad x_w = \text{AE}_w(x, \theta_{A_w}) \quad (5)$$

where $\text{AE}_s(\cdot)$ and $\text{AE}_w(\cdot)$ denote the strong and weak AE, respectively. θ_{A_s} and θ_{A_w} are the parameters of them. The L_2 loss is imposed between the augmented samples and the input time series to achieve the training of the AEs. Specifically, for a time series x , the loss function of the sample augmentation is

$$\text{LossA} = \frac{1}{T} (\|x_s - x\|_2 + \|x_w - x\|_2) \quad (6)$$

where $\|\cdot\|$ denotes the norm.

D. Self-Supervised Contrastive Learning

With the discrepancy in information loss between different AEs, the strong and weak samples are differentiated from each other while containing the shared (inherent) feature. The CLR-DRL extracts the inherent representation of the time series by capturing the shared features of the augmentation samples through self-supervised comparative learning.

The features of the two augmented samples are extracted by a ResNet with shared weights as the representations, which is calculated by

$$z_* = F(x_*, \theta_F) \quad (7)$$

where $F(\cdot)$ is the representation extraction operation of the ResNet and θ_F denotes the shared parameters. The $z_* \in \mathbb{R}^{D_r \times 1} = \{z_s, z_w\}$ is the representation vector of the samples $x_* = \{x_s, x_w\}$ and D_r denotes the dimension of z_* . For the $F(\cdot)$, the calculation of every residual block can be calculated by

$$z_*^n = \text{MaxPool}(\text{BN}(\text{Conv}(z_*^{n-1}))) + z_*^{n-1} \quad (8)$$

where $\text{MaxPool}(\cdot)$, $\text{BN}(\cdot)$, and $\text{Conv}(\cdot)$ denote the max pooling operation, batch normalization operation, and convolution operation, respectively. And when $n = 1$, $z_*^0 = x_*$. The representation z_* is obtained by flattening the z_*^n . Then, the representations z_s and z_w are concatenated to obtain the representation z of the input time series. For a sample x

$$z = \text{concat}(z_s, z_w) \\ = [F(\text{AE}_s(x, \theta_{A_s}), \theta_F); F(\text{AE}_w(x, \theta_{A_w}), \theta_F)] \quad (9)$$

where $\text{concat}(\cdot)$ denote the concatenation function.

Furthermore, the representation z_* is projected into the contrastive loss space via a shared-weight projection head. Current research has shown that it is more beneficial to perform contrastive loss calculations in this space [7]. This calculation can be formulated by

$$h_* = G(z_*, \theta_G) = W_{g2} \cdot \text{ReLU}(\text{BN}(W_{g1} \cdot z_*)) \quad (10)$$

where h_* is the projection of z_* , θ_G is the shared parameters of the projection head, and W_{g1} and W_{g2} are the weight matrices of two linear transformations. In order to maximize the similarity of positive pairs and minimize the similarity of negative pairs, the normalized temperature-scaled cross-entropy loss (NT-Xent) [7] is used as the contrastive loss constrain. Specifically, the loss function of the sample x_i in a small batch can be defined as

$$\text{LossC}(h_s^i, h_w^i) = -\log\left(\frac{\exp(S(h_s^i, h_w^i)/\tau)}{\sum_{h_*^j \in D^{\text{batch}}} \mathbf{1}_{i \neq j} \exp(S(h_s^i, h_w^i)/\tau)}\right) \quad (11)$$

where $S(\cdot)$ and $\mathbf{1}$ denote the cosine similarity and the indicator function, respectively. h_s^i and h_w^i are the augmented samples of the sample x_i , and the h_*^j represents other augmented samples in a small batch. And τ is a temporal parameter.

In summary, the objective function of the CLR-DRL is shown in the following equation:

$$L = \text{LossA} + \text{LossC}. \quad (12)$$

E. Effectiveness of the CLR-DRL

The pretraining process includes the sample augmentation and self-supervised contrastive learning processes. And the two processes are combined for simultaneous training, making the CLR-DRL as an end-to-end method, which is beneficial in fully utilizing the AE to generate augmentation samples while learning the inherent representation of the time series. In order to illustrate the effectiveness of the CLR-DRL in extracting the inherent representation of the input time series, it is analyzed from the deduction of the gradient flow.

First, the objective function of the CLR-DRL is shown in (12). Then, based on the current work [23], the contrast loss constrain in (11) can be simplified as

$$\text{LossC}_{\text{simple}}(x_i) = -S(h_s^i, h_w^i) + \sum_{h_*^j \in D^{\text{batch}}} S(h_*^i, h_*^j). \quad (13)$$

For the two terms contained in (13), there is only a difference in sign. The negative term means maximizing the similarity within positive pairs, and the positive term means minimizing the similarity within negative pairs. Therefore, here only the gradient of the negative term is calculated and analyzed, and the gradient of the positive term can be obtained by a similar approach. Ignoring the positive term in (13), letting $\text{LossC} = \text{LossC}_{\text{simple}}$ and substituting it into (12), the gradients of L with respect to the parameters θ_G , θ_F , θ_{A_s} , and θ_{A_w} can be calculated by the chain rule

$$\frac{\partial L}{\partial \theta_G} = \frac{\partial \text{LossC}}{\partial S} \frac{\partial S}{\partial h_s} \frac{\partial h_s}{\partial \theta_G} + \frac{\partial \text{LossC}}{\partial S} \frac{\partial S}{\partial h_w} \frac{\partial h_w}{\partial \theta_G} \\ = -\frac{\partial \text{LossC}}{\partial S} h_w \frac{\partial h_s}{\partial \theta_G} - \frac{\partial \text{LossC}}{\partial S} h_s \frac{\partial h_w}{\partial \theta_G} \\ = -\frac{\partial \text{LossC}}{\partial S} \left(h_w \frac{\partial h_s}{\partial \theta_G} + h_s \frac{\partial h_w}{\partial \theta_G} \right) \quad (14)$$

$$\frac{\partial L}{\partial \theta_F} = \frac{\partial \text{LossC}}{\partial S} \frac{\partial S}{\partial h_s} \frac{\partial h_s}{\partial z_s} \frac{\partial z_s}{\partial \theta_F} + \frac{\partial \text{LossC}}{\partial S} \frac{\partial S}{\partial h_w} \frac{\partial h_w}{\partial z_w} \frac{\partial z_w}{\partial \theta_F} \\ = -\frac{\partial \text{LossC}}{\partial S} h_w \frac{\partial h_s}{\partial z_s} \frac{\partial z_s}{\partial \theta_F} - \frac{\partial \text{LossC}}{\partial S} h_s \frac{\partial h_w}{\partial z_w} \frac{\partial z_w}{\partial \theta_F} \\ = -\frac{\partial \text{LossC}}{\partial S} \left(h_w \frac{\partial h_s}{\partial z_s} \frac{\partial z_s}{\partial \theta_F} + h_s \frac{\partial h_w}{\partial z_w} \frac{\partial z_w}{\partial \theta_F} \right) \quad (15)$$

$$\frac{\partial L}{\partial \theta_{A_s}} = \frac{\partial \text{LossC}}{\partial S} \frac{\partial S}{\partial h_s} \frac{\partial h_s}{\partial z_s} \frac{\partial z_s}{\partial x_s} \frac{\partial x_s}{\partial \theta_{A_s}} + \frac{\partial \text{LossA}}{\partial x_s} \frac{\partial x_s}{\partial \theta_{A_s}} \\ = -\frac{\partial \text{LossC}}{\partial S} h_w \frac{\partial h_s}{\partial z_s} \frac{\partial z_s}{\partial x_s} \frac{\partial x_s}{\partial \theta_{A_s}} + \frac{\partial \text{LossA}}{\partial x_s} \frac{\partial x_s}{\partial \theta_{A_s}} \\ = \left(-\frac{\partial \text{LossC}}{\partial S} \frac{\partial h_s}{\partial z_s} \frac{\partial z_s}{\partial x_s} h_w + \frac{x_s - x}{T \|x_s - x\|_2} \right) \frac{\partial x_s}{\partial \theta_{A_s}} \quad (16)$$

$$\frac{\partial L}{\partial \theta_{A_w}} = \frac{\partial \text{LossC}}{\partial S} \frac{\partial S}{\partial h_w} \frac{\partial h_w}{\partial z_w} \frac{\partial z_w}{\partial x_w} \frac{\partial x_w}{\partial \theta_{A_w}} + \frac{\partial \text{LossA}}{\partial x_w} \frac{\partial x_w}{\partial \theta_{A_w}} \\ = -\frac{\partial \text{LossC}}{\partial S} h_s \frac{\partial h_w}{\partial z_w} \frac{\partial z_w}{\partial x_w} \frac{\partial x_w}{\partial \theta_{A_w}} + \frac{\partial \text{LossA}}{\partial x_w} \frac{\partial x_w}{\partial \theta_{A_w}} \\ = \left(-\frac{\partial \text{LossC}}{\partial S} \frac{\partial h_w}{\partial z_w} \frac{\partial z_w}{\partial x_w} h_s + \frac{x_w - x}{T \|x_w - x\|_2} \right) \frac{\partial x_w}{\partial \theta_{A_w}}. \quad (17)$$

From (14) and (15), the gradient of the shared parameters θ_G and θ_F includes the gradient of two branches of strong and weak augmented samples. Each of them has a strong connection with the projection term h_* of the other augmented sample, since h_* serves as the weight coefficient of the other term multiplied directly on the gradient. Therefore, according to the purpose of pulling similar samples closer and the updating process of the shared parameter θ_F , the representation vector z_* of the augmented samples x_* extracted by $F(x_*, \theta_F)$ gradually converges to their common representation, i.e., to the inherent representation of the input time series.

Furthermore, from (16) and (17), it is known that in the end-to-end training process, in addition to the constraints of the

AE reconstruction process, the contrast loss term LossC will impose an additional constraint on the augmentation process. With this constraint, an augmented sample will be affected by the projection term of the representation vector of the other sample, since h_* also serves as the weight coefficient of the gradient term. This will cause the common representation between the two augmented samples to gradually increase with the training process, while gradually reducing the differential representation between the two samples.

The above analysis is also proven in this Section on ablation study and representation analysis using alignment and uniformity in Sections IV-F and IV-G, that is, the representation of the positive pair shows well alignment and uniformity. Therefore, it is proved that the CLR-DRL can achieve good extraction of the inherent representation of the input time series.

F. Working on Target Tasks

After pretraining, all parameters $\Theta = \{\theta_{A_s}, \theta_{A_w}, \theta_F, \theta_G\}$ can be obtained. Among them, θ_G is only used to project the representation z_* into the contrastive loss space. Therefore, the parameters of the pretrained model used to extract the representation vector are $\Theta = \{\theta_{A_s}, \theta_{A_w}, \theta_F\}$. Then, combined with the target task, the obtained pretrained model and the target model are combined into a complete model, which is fine-tuned on the target domain to obtain the parameters $\Phi = \{\phi_{A_s}, \phi_{A_w}, \phi_F\}$ as well as the parameter ϕ_T of the target task model F_{target} . Therefore, given the input time series x , the prediction result $\hat{y}$ of the target task is

$$\hat{y} = F_{\text{target}}(z, \phi_T) = F_{\text{target}}(F_{\text{tune}}(x, \Phi), \phi_T). \quad (18)$$

IV. EXPERIMENTS AND ANALYSIS ON BENCHMARKS

In order to fully evaluate the CLR-DRL, experiments are conducted on two typical time series tasks: classification and AD. In an actual industrial, the number of labeled samples required for both tasks is often scarce. Furthermore, these two tasks are suitable choices in terms of validating the motivation of the proposed method and validating its improvement in the class aliasing problem of augmented samples. For each task, experiments are performed in both in- and cross-domain settings, and results are reported in detail.

A. Benchmarks

The CLR-DRL is validated on nine real-world datasets of two typical time series tasks and the details are shown in Table I. Among them, the SleepEEG, the HAR, and the ECG are only used as pretraining datasets and are not segmented. The remaining datasets are split into training, validation, and test sets following the current protocol. Specifically, for classification datasets, the settings remained consistent with the TF-C [6]. For the AD datasets, the settings of the server machine dataset (SMD) are consistent with the literature [24]. The datasets [25] mars science laboratory (MSL) and soil moisture active passive (SMAP) are 27 and 55 channels of telemetry and command data of the two spacecraft, respectively, in which only telemetry dimension data are considered, that is, using them as univariate AD datasets.

B. Baselines

The CLR-DRL is compared with eight competitive pretraining methods, including six contrastive pretraining methods including the TF-C [6], the CoST [16], the TS2Vec [14], the Mixing-up [26], the TS-TCC [9], and the TS-TAC [27], and two mask pretraining methods the TimeMAE [28] and the SimMTM [29]. Mask pretraining is another widely used method, so here two new mask pretraining method is considered for comparison. Among the eight methods, the TF-C is the previous SOTA contrastive pretraining method.

C. Setup

For the time series classification task, settings remained consistent with the TF-C, which is followed by many followed works. Specifically, the scenario Epilepsy $\rightarrow$ Epilepsy is set as an in-domain setting and for cross-domain settings, there are four scenarios including SleepEEG $\rightarrow$ Epilepsy, SleepEEG $\rightarrow$ EMG, HAR $\rightarrow$ Gesture, and ECG $\rightarrow$ electromyography (EMG). The cross-entropy loss is used as a constraint on the classification task to update parameters during fine-tuning.

For the AD task, the following in-domain scenarios are set up: MSL-A-4 $\rightarrow$ All, SMAP-P-1 $\rightarrow$ All, and SMD-2-1 $\rightarrow$ All, which means the pretraining is conducted on a certain entity or telemetry channel, and fine-tuning as well as AD are conducted on all entities and telemetry channels in the same dataset. Under the cross-domain setting, the following scenarios are set up: MSL-A-4 $\rightarrow$ SMAP and SMAP-P-1 $\rightarrow$ MSL. In addition, the widely used AD approach is used, which is that a time series reconstruction model is modeled and AD is achieved using the reconstruction error and a predefined threshold [24]. Regarding the selection of the predefined threshold, the best threshold is selected through grid search, and the best performance is reported for all experiments [30].

For all experimental settings, following the current protocol [6], the target task model F_{target} is implemented using a linear classifier. The experiment results are evaluated using standard evaluation metrics including accuracy ($\text{Acc} = ((\text{TP} + \text{TN})/(\text{TP} + \text{FP} + \text{TN} + \text{FN}))$), precision ($P = (\text{TP}/(\text{TP} + \text{FP}))$), recall ($R = (\text{TP}/(\text{TP} + \text{FN}))$), and $F1$ score ($F1 = ((2 \times P \times R)/(P + R))$), where TP and FP denote correctly and incorrectly detected anomalies, respectively. And TN and FN denote correctly and incorrectly labeled normal, respectively.

D. Implementation Details

The CLR-DRL, its variants, and all baselines are implemented in PyTorch version 1.8.2 with CUDA 10.2. All experiments are conducted on a server with Intel¹ Xeon¹ Silver 4210 CPU at 2.20 GHz and a single NVIDIA TITAN V 12 GB GPU. Benefiting from the simplicity of the design of the CLR-DRL, only four hyper-parameters need to be set, which are the hidden dimensions D_s and D_w of the strong and weak AEs, the encode layer n of the AE_s and the temperature coefficient τ in the contrast loss. For all experimental scenarios and settings, the D_s , D_w , n , and τ are set to 16, half the sequence length

¹Registered trademark.

TABLE I
DETAILS OF BENCHMARKS

Tasks	Datasets		Channels	Length	Samples	Cls. or AR.	Information	Freq.(Hz) or Entities
Classification	SleepEEG		1	200	371005/-/-	5	EEG	100
	HAR		9	128	10299/-/-	6	Activity	50
	ECG		1	1500	43673/-/-	4	ECG	300
	Epilepsy		1	178	60/20/11420	2	EEG	174
	Gesture		3	315	320/120/120	8	Hand movement	100
	EMG		1	1500	122/41/41	3	Muscle responses	4K
Anomaly Detection		A-4			2647/20/8064			
	MSL	Others	55(1)	16	44501/11125/65665	0.105	Spacecraft	27
		P-1			2856/20/8489			
	SMAP	Others	25(1)	16	105861/26465/419128	0.128	Spacecraft	55
		M-2-1			23677/20/23678			
	SMD	Others	38	8	547728/136945/684742	0.042	Server	28

TABLE II
CLASSIFICATION PERFORMANCE (%) OF IN-DOMAIN AND CROSS-DOMAIN SETTINGS

Scenarios	In-Domain				Corss-Domain															
	Epilepsy→Epilepsy				SleepEEG→Epilepsy				SleepEEG→EMG				HAR→Gesture				ECG→EMG			
Model	Acc.	P	R	F1	Acc.	P	R	F1	Acc.	P	R	F1	Acc.	P	R	F1	Acc.	P	R	F1
Random init	93.51	93.87	85.21	88.71	80.21	40.11	50.00	44.51	46.34	15.45	33.33	21.11	72.50	69.98	72.50	70.46	46.34	15.45	33.33	21.11
CLR-DRL	96.32	95.27	92.96	94.06	95.37	93.56	91.60	92.54	97.56	98.33	98.04	98.14	82.50	83.37	82.50	81.26	100.0	100.0	100.0	100.0
TF-C	94.86	93.48	89.93	91.57	94.95	94.56	89.08	91.49	81.71	72.65	81.59	76.83	78.24	79.82	80.11	79.91	97.56	94.44	98.03	95.96
CoST	84.20	89.55	60.47	62.85	82.79	73.83	63.86	66.33	63.41	57.50	57.08	65.86	58.33	58.98	58.33	57.24	92.68	87.50	94.53	89.45
TS2Vec	84.19	79.27	64.36	67.46	93.95	90.59	90.39	90.45	78.54	80.40	67.85	67.66	64.53	62.87	64.51	62.61	97.04	96.66	97.51	97.46
Mixing-up	91.45	94.98	78.51	83.75	80.21	40.11	50.00	44.51	30.24	10.99	25.83	15.41	71.83	70.01	71.83	69.91	91.21	80.07	85.18	82.15
TS-TCC	92.04	95.23	80.02	85.10	92.53	94.51	81.81	86.33	78.89	58.51	63.10	59.04	75.93	76.68	75.66	74.57	95.90	96.84	89.94	92.44
TimeMAE	83.83	80.53	62.14	64.89	87.22	82.28	74.12	77.10	58.54	54.32	45.61	43.71	60.83	55.73	60.83	57.24	95.12	92.69	96.28	94.21
SimMTM	95.25	93.64	91.11	92.30	94.81	91.41	92.46	91.92	<u>95.12</u>	<u>96.28</u>	<u>96.28</u>	<u>96.28</u>	76.67	76.53	76.67	74.86	97.56	98.33	98.04	98.14
TS-GAC	92.62	89.90	86.07	87.80	91.79	91.15	81.91	85.50	70.73	69.84	72.59	70.75	75.83	76.13	75.83	73.96	90.24	88.67	92.57	90.35
$\Delta \uparrow$	1.07	0.04	1.85	1.76	0.42	-1.00	-0.86	0.62	2.44	2.05	1.76	1.86	4.26	3.55	2.39	1.35	2.44	1.67	1.96	1.86

($T/2$), 2, and 0.2, respectively. In addition, for fairness, the parameter settings of the three-layer ResNet are kept consistent with current work [6] to verify the effectiveness of the motivation and architecture of the CLR-DRL. All models are trained using the Adam optimizer with the learning rate 2×10^{-3} , $L2$ regularization penalty coefficient weight 3×10^{-3} and the exponential decay rates hyperparameters β_1 and β_2 for the moment estimates as 0.9 and 0.999. For the classification task, pretraining iterations 100 times and fine-tuning 100 times. For the AD task, pretraining iterations 30 times and fine-tuning 30 times. The batch size of pretraining and fine-tuning for all experiments are set to 128 and 60, respectively. The model and evaluation metrics corresponding to the best $F1$ are recorded and reported.

In addition, for fairness, a unified target task model F_{target} is used for all experiments. Specifically, a two-layer FNN is used to implement target tasks, in which the output of the first layer is active by sigmoid function, and the second layer is the output layer. For classification and AD tasks, the number of neurons in the first layer is 64 and 512, respectively. The number of neurons in the second layer is the number of categories and the length of the input sequence, respectively. Furthermore, all baselines are implemented by their official code, and hyper-parameters are set as they recommended.

E. Results and Discussion

This section reports the main comparison results of two tasks. Experiment results of fine-tuning parameters with random initialization (Random init.) directly on the target dataset without pretraining are also reported.

1) *Classification*: The classification performance for all settings is reported in Table II. Compared with all baselines, the CLR-DRL achieves the best performance on 22 out of 24 metrics. In particular, in all settings, the performance of the CLR-DRL exceeds that of the randomly initialized fine-tuned model, indicating that the proposed model does not suffer from a negative transfer phenomenon, further illustrating that the inherent representation learned by the CLR-DRL during the pretraining is valuable for target tasks. Specifically, under an in-domain setting, compared with the previous SOTA method with the SimMTM, the CLR-DRL achieves a performance improvement of 1.76% on $F1$. From cross-domain settings results, even though there is a large domain gap between the source and target dataset, the CLR-DRL still achieves well classification accuracy compared to baselines. Particularly, in the SleepEEG $\rightarrow$ EMG scenario, the CLR-DRL achieves an improvement of 21.31% on the $F1$ compared with the previous SOTA contrastive learning-based method TF-C. It can be found that when the series length and knowledge of the target domain and the source domain are close, the CLR-DRL can obtain better classification performance. Specifically, in the ECG $\rightarrow$ EMG scenario, the series lengths of two datasets are the same, and the CLR-DRL achieves completely accurate classification performance.

2) *Anomaly Detection*: The AD performance for all settings is reported in Table III. Compared with all baselines, the CLR-DRL achieves the best performance on 18 out of 20 metrics. Similar to the classification task, all metrics of the CLR-DRL outperform the Random init. In in-domain settings, the CLR-DRL achieves significant improvements in

TABLE III
AD PERFORMANCE (%) OF IN-DOMAIN AND CROSS-DOMAIN SETTINGS

Scenarios	In-Domain												Cross-Domain							
	MSL-A-4→All				SMAP-P-1→All				SMD-2-1→All				MSL-A-4→SMAP				SMAP-P-1→MSL			
Model	Acc.	P	R	F1	Acc.	P	R	F1	Acc.	P	R	F1	Acc.	P	R	F1	Acc.	P	R	F1
Random init.	97.39	88.90	99.80	90.27	96.57	88.97	99.67	90.45	99.26	92.19	86.68	87.02	98.46	88.58	95.85	90.97	96.84	86.47	98.63	87.95
CLR-DRL	99.73	98.18	99.03	98.46	99.92	99.25	99.88	99.55	99.66	94.70	93.40	93.65	99.33	96.46	100.0	96.88	99.29	96.69	99.03	97.31
TF-C	98.19	89.23	97.54	91.35	96.98	89.88	100.0	90.74	98.96	87.19	89.22	86.31	96.05	87.49	99.88	88.50	95.84	85.82	99.03	87.32
CoST	95.48	84.95	100.0	87.14	97.28	92.08	99.71	92.44	98.87	88.09	73.03	76.63	97.85	92.28	100.0	92.89	96.21	87.01	99.03	88.40
TS2Vec	96.30	86.26	99.03	87.81	97.32	90.77	99.38	91.47	98.74	86.30	77.26	76.99	97.81	90.72	99.72	91.83	96.28	86.03	99.03	87.67
Mixing-up	95.73	83.71	99.03	86.03	97.90	93.12	100.0	93.97	98.73	86.48	76.46	77.09	97.68	91.27	100.0	92.18	95.79	83.81	99.03	86.12
TS-TCC	95.47	78.30	97.54	80.68	96.30	87.79	98.80	89.38	98.58	85.84	72.52	74.21	95.84	85.19	98.76	87.62	95.57	79.59	97.54	81.48
TimeMAE	96.42	84.33	97.71	87.11	92.86	78.68	98.01	80.32	99.13	87.25	91.13	88.14	93.17	78.25	98.01	80.09	97.20	84.91	96.22	87.40
SimMTM	95.71	83.27	99.03	85.70	97.89	93.27	100.0	94.06	99.18	89.82	91.29	89.66	97.86	91.57	99.71	92.58	95.65	82.97	99.03	85.57
TS-GAC	95.74	85.37	99.03	87.11	97.90	93.42	99.39	93.81	98.15	87.40	83.56	81.43	97.73	91.14	99.72	92.19	95.75	84.80	99.03	86.68
$\Delta \uparrow$	1.54	8.95	-0.97	7.11	2.02	6.13	-0.12	5.49	0.48	4.88	2.11	3.99	1.47	4.18	0.00	3.99	2.09	9.68	0.00	8.91

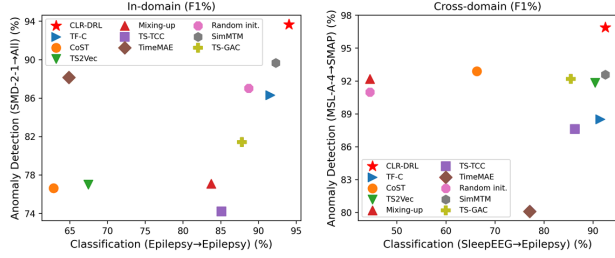


Fig. 3. Performance ($F1$ metric) in classification ($\uparrow$) and AD ($\uparrow$) tasks under in-domain and cross-domain settings.

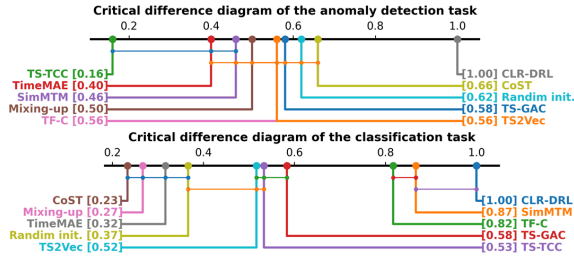


Fig. 4. Critical difference diagram ($F1$) of AD and classification tasks.

both multivariate and univariate AD tasks compared with previous SOTA results. On the $F1$ metric, the CLR-DRL improved by 7.11% on MSL-A-4 $\rightarrow$ All, 5.58% on SMAP-P-1 $\rightarrow$ All, and 3.98% on SMD-2-1 $\rightarrow$ All. In addition, the CLR-DRL also achieves significant improvements in the cross-domain transfer tasks: 3.99% on MSL-A-4 $\rightarrow$ SMAP and 8.91% on SMAP-P-1 $\rightarrow$ MSL.

From the methodological and technical perspective, the well AD performance of the CLR-DRL can be attributed to the following two reasons. First, the CLR-DRL does not use an explicit augmentation approach. Instead, it uses the AEs to automatically obtain augmented samples, which mitigates the impact of category aliasing on the performance of target tasks. Particularly, AD is sensitive to category aliasing and sample changes, and the design of the CLR-DRL is more suitable for such a task. Second, the augmentation samples of the CLR-DRL are the series reconstructed by AEs, so the representation of the series extracted by the CLR-DRL is relatively suitable for the reconstruction task, which has inherent advantages for the reconstruction error-based AD method used by the CLR-DRL.

3) *Summary*: The $F1$ metrics of four scenarios in the in- and cross-domain settings are summarized in Fig. 3 and the visualization of the critical difference diagram based on the Wilcoxon pair-wise signed-rank test after rejecting the null hypothesis using the Friedman test of the metric $F1$ is shown in Fig. 4. In all these settings, the CLR-DRL exhibits good performance, which shows that previous methods cannot simultaneously handle classification and AD tasks in multiple settings, further highlighting the advantages of the CLR-DRL.

All in all, the high performance in the in-domain setting demonstrates the positive effect of extracting time series inherent representation based on the CLR-DRL for target tasks. The high performance in cross-domain settings demonstrates its advantage in achieving high-quality target tasks through cross-domain transfer in scenarios where real-world labeled samples are scarce.

F. Ablation Study

The motivation of the CLR-DRL is investigated through three different ablations, and the high-quality inherent representation of the time series can be extracted by contrastive pretraining with sample augmentation from AE. Specifically, three widely used time series explicit augmentation approaches including jittering, scaling, and permutation are used to replace the AE of the CLR-DRL to construct three variants W/o AE_s (replace AE_s), W/o AE_w (replace AE_w), and W/o AE (replace AE_s , and AE_w). Ablation experiments are conducted on four scenarios including in- and cross-domain settings, and the best performance of the variants under three explicit augmentation approaches are reported in Fig. 5. In addition, Random init. is also considered as a method of ablation and included for comparison.

It can be seen from the results that after replacing any AE with explicit approach, the $F1$ will decline to varying degrees. This is because the correlation between the two augmented samples is weakened, which is detrimental to extracting the inherent representation of the series. This result further supports the theoretical analysis in Section III-E. Furthermore, it is worth noting that after completely replacing the AE with an explicit augmented approach, negative transfer occurs in the in-domain setting. And for the SleepEEG $\rightarrow$ Epilepsy scenario, its performance remains consistent with Random init., which shows that when AEs are not used to augment

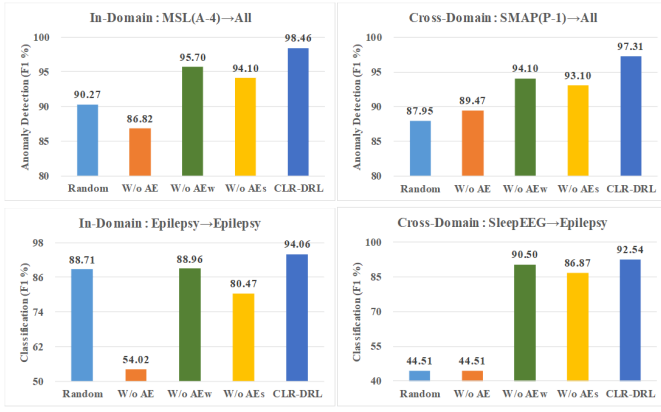


Fig. 5. Ablation results of the CLR-DRL in two tasks and two settings.

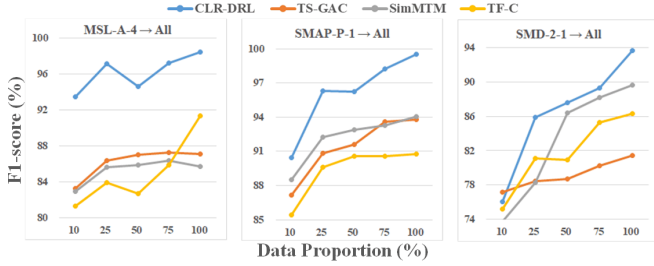


Fig. 6. Performance comparison on fine-tuning with limited data in AD task under in-domain setting.

samples, the variants are difficult to obtain effective domain knowledge and extract representation through pretraining, and cannot provide a positive effect for target tasks. In addition, it is found that in four scenarios, removing strong AE will produce more performance degradation than removing weak AE, which shows that the reconstruction performance of the AE has an impact on the target task performance of the CLR-DRL. This phenomenon will be discussed in Section IV-G.

The above results and analysis prove the correctness of the motivation of the CLR-DRL.

G. Model Analysis

1) *Fine-Tune With Limited Data*: In real-world industrial scenarios, due to the scarcity of labeled samples, transferring pretrained models to limited data scenarios is a key application. Particularly, for typical unsupervised AD tasks, monitoring newly used industrial devices is crucial. In order to verify the performance of the CLR-DRL in fine-tuning on limited data, experiments are conducted in three AD scenarios. As shown in Fig. 6, in the univariate AD task, the performance of the CLR-DRL has been significantly improved compared with baselines under different amounts of data. And the CLR-DRL only needs 10% of the data to obtain more than 90% of $F1$. For multivariable detection tasks, although the CLR-DRL and baselines have similar performance under 10% of the data, as the ratio increases, the CLR-DRL shows obviously rapid performance improvement. At a data ratio of 25%, the CLR-DRL can achieve an AD performance of more than 85% on $F1$, which fully validates the advantages of the CLR-DRL.

2) *Representation Analysis*: In order to intuitively evaluate the representation capabilities of the CLR-DRL, the alignment,

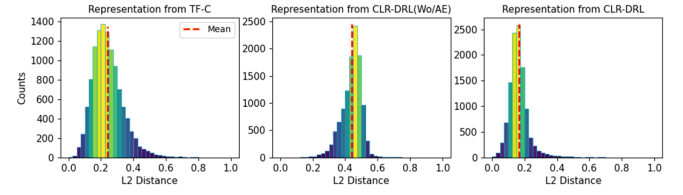


Fig. 7. $L2$ distance distribution of aligned positive-pair representations. A low mean value indicates well alignment.

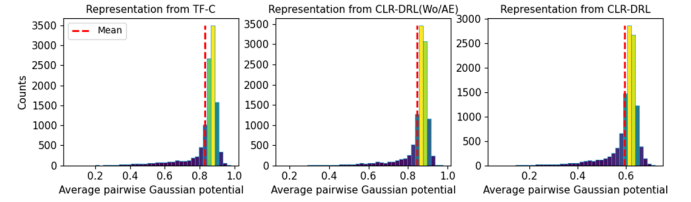


Fig. 8. Average pairwise Gaussian potential of positive-pair representations. A low mean value indicates well uniformity.

and uniformity of representation are used to analyze it. According to Wang and Isola [31], alignment means that the representations of positive pairs should be close to each other and therefore, invariant to noise. Uniformity means that the representation of all samples should be uniformly distributed on the hypersphere, which allows it to retain more data information. For a good representation, it should have both good alignment and uniformity, and they have a direct relationship with the performance of target tasks. The quantitative metrics proposed by Wang and Isola [31] are used to evaluate the representation extracted by the CLR-DRL, and quantitative results of alignment and uniformity in the SleepEEG → Epilepsy scenario are provided in Figs. 7 and 8.

For alignment, compared with the TF-C and the variant W/o AE, the CLR-DRL has the minimal mean $L2$ distance of the positive pair representation, which indicates that the representation obtained by the CLR-DRL has good alignment. Meanwhile, although the representation of the TF-C is closer to the CLR-DRL in terms of the mean value (0.16 versus 0.24), the representation distribution of the CLR-DRL is more concentrated on short distances in terms of distance distribution. The variant W/o AE has a larger mean distance of 0.44, which shows that generating positive pairs through AEs can lead to better alignment of the representation. For uniformity, the CLR-DRL obtained the minimal mean of the average pairwise Gaussian potential among the three models (0.59 versus 0.82 and 0.84), which shows that it achieves better uniformity and retains the information of the data to the greatest extent while extracting the inherent representation vector of the time series. Therefore, from the perspective of representation analysis, it is clearly shown why the CLR-DRL achieves advantages in target tasks. The small alignment further verifies the conclusion in Section III-E that the two augmented samples are closely related.

3) *Efficiency Analysis*: The number of parameters and the floating point operations (FLOPs) [32] are used to evaluate the memory and computation cost of the CLR-DRL. Specifically, the above metrics are calculated using the experiment settings on the SMD dataset detailed in Sections IV-A and IV-D, and results are shown in Fig. 9. A higher $F1$ score

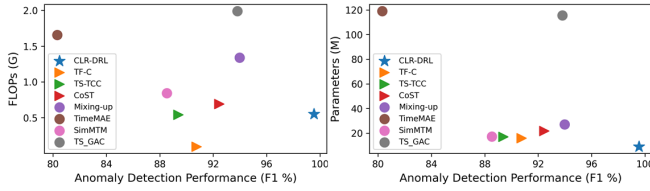


Fig. 9. Relationships between the AD performance ($F1 \uparrow$) and model efficiency (FLOPs $\downarrow$ or parameters $\downarrow$).

TABLE IV

PARAMETERS OF THE NAPHTHA AND LPG PIPELINES

PipeLine	Length (km)	Leak Position(km)	Velocity (m/s)	Pressure (MPa)	
				Upstream	Downstream
Naphtha	15.511	9.476	1055	2.1-RTU03	0.5-RTU04
LPG	14.224	14.224	860	0.93-RTU05	0.35-RTU06

TABLE V
DETAILS OF PSM DATASET

Tasks	Datasets	Samples	Cls. or AR	Pipeline
Cls	RTU03 Pre F-T	17000/-	1	Naphtha
		60/20/766	2	
		60/20/590	2	
		17000/-	1	LPG
AD	RTU03	13600/3400/55000	0.007	Naphtha
	RTU04	13600/3400/55000	0.005	
	RTU05	17000/-	0	LPG

* The Cls, AD, Pre, F-T and AR are the classification, anomaly detection, pre-train, fine-tune and anomaly ratio, respectively.

means a better AD performance, a lower FLOPs means the less amount of computation required by the model (better computation efficiency) and fewer parameters means better memory cost. It is notable that the CLR-DRL achieves the best AD performance and the lowest memory cost. Although the TF-C achieves the best computation efficiency, it cannot detect anomalies well. Instead, the CLR-DRL can take into account both good computation efficiency (rank 2) and model performance. In addition, all models can predict a single sample in less than 1 ms, which is fully sufficient for actual industrial applications. Therefore, The CLR-DRL can achieve the target task efficiently and quickly in a real-world application.

V. VALIDATION ON A REAL-WORLD SCENARIO

Pipeline transportation is a widely used transportation approach for petrochemical materials and products, and plays an important role in a country's petrochemical industry. However, due to aging, corrosion, man-made damage, and other factors, pipeline leakage accidents occur frequently. Therefore, real-time and accurate pipeline status monitoring (PSM) is of great significance. For long-distance pipelines, pressure or acoustic wave detection instruments are usually installed on upstream and downstream of the pipeline to collect the transportation signal, which is used to monitor the pipeline.

In order to verify the effectiveness of the CLR-DRL in a practical industrial application, monitoring on two pipelines of naphtha and liquefied petroleum gas (LPG) under different working conditions as shown in Fig. 10 is conducted. The data are the upstream and downstream infrasound signals collected by the remote terminal unit (RTU) through the infrasound acoustic transducer. The signal is sampled at a frequency of 50 Hz, for which a 1:10 extraction is conducted and the

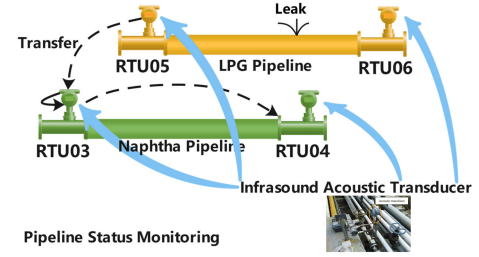


Fig. 10. PSM and experiment settings.

signal is segmented through a time window of length 16. Following the above operation, four consecutive hours of data are used to compose the PSM dataset. In particular, the naphtha pipeline simulated leaks 11 times by opening valves during this period.

The PSM experiment is conducted on two pipelines with different working conditions as shown in Table IV. In experimental settings, the data collected from upstream of the two pipelines are used to pretrain the model and then are transferred to the naphtha pipeline to monitor. The tasks of transferring from one pipeline to another are considered as cross-domain setting and four scenarios containing in- and cross-domain settings shown in Table V are set up as: RTU03 $\rightarrow$ RTU03, RTU03 $\rightarrow$ RTU04, RTU05 $\rightarrow$ RTU03, and RTU05 $\rightarrow$ RTU04. Among them, for typical AD tasks, modeling is conducted on the data under normal working conditions. For the classification task, two leaked data and a small number of normal samples are used to form a fine-tuning dataset to fine-tune the pretrained model and test it on the remaining data. The implementation details are the same as Section IV-D.

The experiment results of AD and classification tasks on the PSM are shown in Tables VI and VII, respectively. It can be seen that the CLR-DRL exceeds the baselines in all settings. Especially in scenario RTU03 $\rightarrow$ RTU04 of the AD task, the CLR-DRL achieves completely accurate AD results and an improvement of 6.64% (93.36 $\rightarrow$ 100.00) is achieved compared to the previous SOTA results. In addition, on the classification task, the CLR-DRL achieves significant improvements in four scenarios. Especially in the two cross-domain scenarios, an improvement of more than 10% is achieved (71.68 $\rightarrow$ 81.98 and 80.67 $\rightarrow$ 90.97).

Furthermore, for actual application scenarios, the monitoring experiments on newly used devices are simulated. In cross-domain scenarios, the AD models are fine-tuned on different proportions of data to monitor the pipeline. As shown in the results in Fig. 11, even if only 10% of the data is used for fine-tuning, more than 85% of the monitoring performance can be achieved. And as data from new devices continues to increase, monitoring performance continues to improve. This shows that the CLR-DRL has a well transfer capability and can play an important role in monitoring new devices in real-world industrial scenarios.

In summary, according to the actual application of the CLR-DRL on PSM, the proposed method has practical significance in ensuring the stable operation of the device and avoiding economic losses and personal injuries.

TABLE VI
AD PERFORMANCE (%) ON PSM DATASET

Scenarios	In-Domain								Cross-Domain							
	RTU03→RTU03				RTU03→RTU04				RTU05→RTU03				RTU05→RTU04			
Model	Acc.	P	R	F1	Acc.	P	R	F1	Acc.	P	R	F1	Acc.	P	R	F1
Random init.	99.91	95.43	87.21	91.13	99.93	100.0	87.12	93.12	99.91	92.51	83.73	87.90	99.93	96.18	85.42	90.48
CLR-DRL	100.0	100.0	99.74	99.87	100.0	100.0	100.0	100.0	<u>99.91</u>	97.62	94.78	96.18	<u>99.92</u>	99.60	93.21	96.30
TF-C	99.93	95.73	93.73	94.72	99.92	100.0	85.42	92.14	99.91	96.90	89.82	93.22	99.91	94.57	88.47	91.42
CoST	99.85	90.05	87.47	88.74	99.78	96.76	60.68	74.58	99.76	94.66	69.45	80.12	99.75	75.40	80.00	77.63
TS2Vec	99.93	99.62	87.80	93.33	99.88	95.13	86.68	90.71	99.87	91.44	89.30	90.36	99.93	99.24	88.14	93.36
Mixing-up	99.88	<u>95.95</u>	86.68	91.08	99.93	99.24	88.14	93.36	99.88	95.68	86.68	90.96	99.93	<u>99.62</u>	88.14	<u>93.53</u>
TS-TCC	99.89	96.51	86.68	91.33	99.93	99.23	87.46	92.97	99.89	96.51	86.68	91.33	99.93	99.23	87.46	92.97
TimeMAE	99.88	95.95	86.68	91.08	99.93	99.24	88.14	93.36	99.88	95.68	86.68	90.96	99.93	98.86	88.14	93.19
SimMTM	99.92	94.71	93.47	94.09	99.92	98.84	86.44	92.22	99.92	95.45	93.21	<u>94.32</u>	99.93	99.61	87.12	92.95
TS-GAC	99.89	96.51	86.68	91.33	99.93	99.62	87.80	93.33	99.89	90.89	<u>93.73</u>	<u>92.29</u>	99.93	100.0	87.12	93.12
$\Delta \uparrow$	0.07	0.38	6.01	5.15	0.07	0.00	11.86	6.64	-0.01	0.72	1.57	1.86	-0.01	-0.02	4.74	2.77

TABLE VII
CLASSIFICATION PERFORMANCE (%) ON PSM DATASET

Scenarios	In-Domain								Cross-Domain							
	RTU03→RTU03				RTU03→RTU04				RTU05→RTU03				RTU05→RTU04			
Model	Acc.	P	R	F1	Acc.	P	R	F1	Acc.	P	R	F1	Acc.	P	R	F1
Random init.	66.71	67.15	65.59	65.40	84.60	87.58	78.80	81.08	65.80	67.58	64.12	63.24	84.46	89.05	77.99	80.51
CLR-DRL	82.51	<u>83.17</u>	83.08	82.51	92.95	94.10	90.43	91.90	81.98	82.24	82.37	81.98	92.17	94.65	88.68	90.79
TF-C	73.24	75.89	71.79	71.58	76.50	74.58	71.55	72.51	63.71	79.88	60.62	54.93	79.37	79.01	73.83	75.27
CoST	59.01	78.40	55.52	46.18	67.75	83.49	53.40	46.47	56.27	55.78	55.66	55.61	66.19	61.38	59.58	59.82
TS2Vec	71.67	80.08	69.45	68.01	80.81	87.24	72.62	74.76	71.93	80.23	69.73	68.37	80.68	86.84	72.52	74.63
Mixing-up	75.98	83.13	74.06	73.55	84.60	88.65	78.36	80.82	73.11	75.11	71.79	71.68	83.94	85.38	78.75	80.67
TS-TCC	61.75	79.25	58.50	51.44	57.96	59.86	60.75	<u>57.57</u>	45.30	46.06	46.31	44.83	47.00	49.40	49.35	46.69
TimeMAE	55.87	57.26	52.82	45.82	73.11	83.90	61.31	60.04	54.44	77.10	50.57	36.27	53.00	52.60	52.87	51.65
SimMTM	77.42	85.00	75.52	75.16	81.98	84.16	75.74	77.76	76.11	75.97	75.89	75.93	89.82	92.32	85.73	87.91
TS-GAC	<u>71.54</u>	73.82	<u>70.09</u>	<u>69.80</u>	78.33	76.76	73.75	74.76	<u>70.10</u>	<u>70.95</u>	<u>70.73</u>	<u>70.09</u>	<u>89.69</u>	<u>90.98</u>	<u>86.25</u>	<u>87.98</u>
$\Delta \uparrow$	5.09	-1.83	7.56	7.35	8.35	5.45	12.07	11.08	5.87	2.01	6.48	6.05	2.35	2.33	2.95	2.88

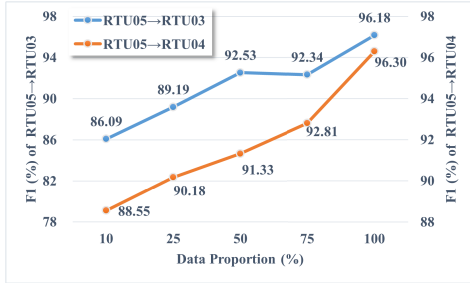


Fig. 11. Performance of the CLR-DRL fine-tuning with limited data in AD task on PSM.

VI. CONCLUSION

This article proposes a new contrastive pretraining method with the CLR-DRL. In order to solve the problem of augmented samples category aliasing that exists in explicit data augmentation approaches, the CLR-DRL uses two different AEs to automatically generate augmented samples and obtain the positive and negative pairs required for contrastive learning. In turn, the reconstruction discrepancies of different AEs as well as the contrastive constraint are utilized to achieve the extraction of inherent representation vectors of the time series, which is explained by the theory analysis using gradient flow calculation. Experiments on nine real-world benchmarks of two typical time series tasks show that the CLR-DRL outperforms the current SOTA baselines on Acc and F1 metrics. In addition, an actual PSM case is provided to verify the actual

application effect of the CLR-DRL. Further analysis proves the correctness of the idea of the CLR-DRL.

However, as a deep learning-based method, the CLR-DRL has poor interpretability, which is also significant in real-world applications. Therefore, in the future, we will conduct research on interpretable machine learning for time series. Meanwhile, we will focus on the correlation between mask modeling and contrastive modeling of time series and conduct research on robust series representation.

REFERENCES

- [1] A. Tank, I. Covert, N. Foti, A. Shojaie, and E. B. Fox, "Neural Granger causality," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 44, no. 8, pp. 4267–4279, Aug. 2022.
- [2] Z. Zhang, Z. Geng, and Y. Han, "Graph structure change-based anomaly detection in multivariate time series of industrial processes," *IEEE Trans. Ind. Informat.*, vol. 20, no. 4, pp. 6457–6466, Apr. 2024.
- [3] Z. Zhang, Y. Han, B. Ma, M. Liu, and Z. Geng, "Temporal chain network with intuitive attention mechanism for long-term series forecasting," *IEEE Trans. Instrum. Meas.*, vol. 72, pp. 1–13, 2023.
- [4] X. Hu, T. Zhang, Z. Geng, and Y. Han, "Noise adaptive filtering model integrating spatio-temporal feature for soft sensor," *Expert Syst. Appl.*, vol. 239, Apr. 2024, Art. no. 122453.
- [5] E. Eldele, M. Ragab, Z. Chen, M. Wu, C.-K. Kwok, and X. Li, "Label-efficient time series representation learning: A review," 2023, *arXiv:2302.06433*.
- [6] X. Zhang, Z. Zhao, T. Tsiligkaridis, and M. Žitnik, "Self-supervised contrastive pre-training for time series via time-frequency consistency," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, Jan. 2022, pp. 3988–4003.
- [7] T. Chen, S. Kornblith, M. Norouzi, and G. E. Hinton, "A simple framework for contrastive learning of visual representations," in *Proc. 37th Int. Conf. Mach. Learn. (ICML)*, Jan. 2020, pp. 1–23.

- [8] C. Ian Tang, I. Perez-Pozuelo, D. Spathis, and C. Mascolo, "Exploring contrastive learning in human activity recognition for healthcare," 2020, *arXiv:2011.11542*.
- [9] E. Eldele et al., "Time-series representation learning via temporal and contextual contrasting," in *Proc. 13th Int. Joint Conf. Artif. Intell. (IJCAI)*, 2021, pp. 2352–2359.
- [10] H. El-Fiqi, M. Wang, K. Kasmarik, A. Bezerianos, K. C. Tan, and H. A. Abbass, "Weighted gate layer autoencoders," *IEEE Trans. Cybern.*, vol. 52, no. 8, pp. 7242–7253, Aug. 2022.
- [11] K. He, X. Chen, S. Xie, Y. Li, P. Dollár, and R. Girshick, "Masked autoencoders are scalable vision learners," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2022, pp. 15979–15988.
- [12] T. B. Brown et al., "Language models are few-shot learners," in *Proc. NIPS*, 2020, pp. 1877–1901.
- [13] J.-Y. Franceschi, A. Dieuleveut, and M. Jaggi, "Unsupervised scalable representation learning for multivariate time series," in *Proc. Adv. Neural Inf. Process. Syst. (NeuralIPS)*, Jan. 2019, pp. 1–23.
- [14] Z. Yue et al., "Ts2vec: Towards universal representation of time series," in *Proc. AAAI Conf. Artif. Intell.*, vol. 36, no. 8, 2022, pp. 8980–8987.
- [15] L. Yang and S. Hong, "Unsupervised time-series representation learning with iterative bilinear temporal-spectral fusion," in *Proc. Int. Conf. Mach. Learn.*, 2022, pp. 25038–25054.
- [16] G. Woo, C. Liu, D. Sahoo, A. Kumar, and S. C. H. Hoi, "CoST: Contrastive learning of disentangled seasonal-trend representations for time series forecasting," in *Proc. ICLR*, Jan. 2022, pp. 1–14.
- [17] T. T. Um et al., "Data augmentation of wearable sensor data for Parkinson's disease monitoring using convolutional neural networks," in *Proc. 19th ACM Int. Conf. Multimodal Interact.*, Nov. 2017, pp. 216–220.
- [18] Q. Wen et al., "Time series data augmentation for deep learning: A survey," in *Proc. Thirtieth Int. Joint Conf. Artif. Intell.*, Aug. 2021, pp. 4653–4660.
- [19] Y. Shi, J. Li, H. Li, and B. Yang, "An imbalanced data augmentation and assessment method for industrial process fault classification with application in air compressors," *IEEE Trans. Instrum. Meas.*, vol. 72, pp. 1–10, 2023.
- [20] Y. Li, X. Lu, Y. Wang, and D. Dou, "Generative time series forecasting with diffusion, denoise, and disentanglement," in *Proc. Adv. Neural Inf. Process. Syst. (NeuralIPS)*, Jan. 2023, pp. 23009–23022.
- [21] L. Lin, Z. Li, R. Li, X. Li, and J. Gao, "Diffusion models for time series applications: A survey," 2023, *arXiv:2305.00624*.
- [22] P. Yan et al., "A comprehensive survey of deep transfer learning for anomaly detection in industrial time series: Methods, applications, and directions," 2023, *arXiv:2307.05638*.
- [23] F. Wang and H. Liu, "Understanding the behaviour of contrastive loss," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2021, pp. 2495–2504.
- [24] Y. Su, Y. Zhao, C. Niu, R. Liu, W. Sun, and D. Pei, "Robust anomaly detection for multivariate time series through stochastic recurrent neural network," in *Proc. 25th ACM SIGKDD Int. Conf. Knowl. Disc. Data Min.*, 2019, pp. 2828–2837.
- [25] K. Hundman, V. Constantinou, C. Laporte, I. Colwell, and T. Söderström, "Detecting spacecraft anomalies using LSTMs and nonparametric dynamic thresholding," in *Proc. 24th ACM SIGKDD Int. Conf. Knowl. Disc. Data Min.*, Jul. 2018, pp. 387–395.
- [26] K. Wickstrøm, M. Kampffmeyer, K. Ø. Mikalsen, and R. Jenssen, "Mixing up contrastive learning: Self-supervised representation learning for time series," *Pattern Recognit. Lett.*, vol. 155, pp. 54–61, Mar. 2022.
- [27] Y. Wang et al., "Graph-aware contrasting for multivariate time-series classification," in *Proc. AAAI Conf. Artif. Intell.*, Mar. 2024, vol. 38, no. 14, pp. 15725–15734.
- [28] M. Cheng, Q. Liu, Z. Liu, H. Zhang, R. Zhang, and E. Chen, "TimeMAE: Self-supervised representations of time series with decoupled masked autoencoders," 2023, *arXiv:2303.00320*.
- [29] J. Dong, H. Wu, H. Zhang, Z. Li, J. Wang, and M. Long, "SimMTM: A simple pre-training framework for masked time-series modeling," in *Proc. Adv. Neural Inf. Process. Syst.*, Jan. 2023, pp. 29996–30025.
- [30] Z. Chen, D. Chen, X. Zhang, Z. Yuan, and X. Cheng, "Learning graph structures with transformer for multivariate time-series anomaly detection in IoT," *IEEE Internet Things J.*, vol. 9, no. 12, pp. 9179–9189, Jun. 2022.
- [31] T. Wang and P. Isola, "Understanding contrastive representation learning through alignment and uniformity on the hypersphere," in *Proc. 37th Int. Conf. Mach. Learn.*, vol. 119, Jul. 2020, pp. 9929–9939.
- [32] P. Molchanov, S. Tyree, T. Karras, T. Aila, and J. Kautz, "Pruning convolutional neural networks for resource efficient inference," in *Proc. 5th Int. Conf. Learn. Represent.*, Jan. 2016, pp. 1–12.



Zhen Zhang received the B.S. degree in measurement and control technology and instrumentation and the M.S. degree in control science and engineering from Beijing University of Chemical Technology, Beijing, China, in 2018 and 2021, respectively, where he is currently pursuing the Ph.D. degree in control science and engineering.

His research interests include time series anomaly detection, forecasting, and industrial process monitoring.



Yongming Han (Member, IEEE) received the B.Sc. and Ph.D. degrees from Beijing University of Chemical Technology, Beijing, China, in 2009 and 2014, respectively.

He is currently a Professor with the College of Information Science and Technology, Beijing University of Chemical Technology. His current research interests include power system modeling, neural networks, intelligent computing, data mining, and intrusion detection.



Bo Ma received the B.S. degree in mechatronic engineering from Taiyuan University of Technology, Taiyuan, China, in 1999, and the Ph.D. degree in power engineering and engineering thermophysics from Beijing University of Chemical Technology (BUCT), Beijing, China, in 2006.

He is currently a Professor with the Diagnosis and Self-Recovery Engineering Research Center, BUCT. His research interests include fault diagnosis, transfer learning, and health monitoring.



Zhiqiang Geng received the B.Sc. and M.Sc. degrees from Zhengzhou University, Zhengzhou, China, in 1997 and 2002, respectively, and the Ph.D. degree from the College of Information Science and Technology, Beijing University of Chemical Technology, Beijing, China, in 2005.

He is currently a Professor with the College of Information Science and Technology, Beijing University of Chemical Technology. His research interests include neural networks, intelligent computing, data mining, knowledge management, and process modeling.